

Reviewer Pack — Minimal Rank of Algebraic Topology Invariants vs SOS Degree ...

Entry 9d3b0b0b8113 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 08:48:15 UTC

Minimal Rank of Algebraic Topology Invariants vs SOS Degree for Max-CUT

Entry ID: 9d3b0b0b8113 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 08:48:15 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Configuration Space Theory) **Field B** (complexity object): Complexity Theory: Sum-of-squares Lower Bounds for Max-CUT

Statement:

[‘For any Max-CUT instance with $n \leq 40$, the minimum rank of its configuration space invariant is asymptotically equivalent to the SOS degree required for a 0.879-approximation.’, ‘More formally, let $I(n)$ be the set of all configurations of a Max-CUT instance on n vertices, and let $C(I(n))$ be the configuration space of $I(n)$. Then, there exists a constant c such that $\min_r \text{rank}(C(I(n))) = \Theta(\text{SOS_degree}(c n \log n))$ for all $n \leq 40$.’]

Rationale (proposer’s reasoning):

[‘This conjecture links algebraic topology with sum-of-squares (SOS) complexity theory. Configuration space invariants, which are well-studied in algebraic topology, could provide a new avenue to analyze the complexity of Max-CUT.’, ‘If true, it would suggest that configuration space invariants can be used to understand the SOS degree required for approximation algorithms, potentially leading to new insights into the approximability of Max-CUT.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 085e9e167a08c38b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Max-CUT instance with $n \leq 40$, if the minimum rank of its configuration space invariant is within a factor of 2 from the SOS degree required for a 0.879-approximation across all seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "configuration space invariant" AND "SOS degree" AND Max-CUT - "algebraic topology" AND "sum-of-squares lower bounds" AND "Max-CUT approximation" - "minimal rank" IN algebraic topology AND "SOS degree" IN complexity theory AND Max-CUT

Top relevant hits considered: - [http://arxiv.org/abs/math/0502195v2] Hopf algebra structure on topological Hochschild homology - [http://arxiv.org/abs/1612.03133v3] Topological Complexity of the Klein bottle - [http://arxiv.org/abs/math/0302340v2] Pure homology of algebraic varieties

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.append((i, j))
        return edges

    def configuration_space_invariant(edges):
        # Placeholder implementation
        return len(edges)

    def sos_degree(n):
        # Placeholder implementation
        return int(2 * math.log2(n) + 1)

    n = random.randint(5, 40)
    instance = generate_max_cut_instance(n)
    invariant_rank = configuration_space_invariant(instance)
    sos_deg = sos_degree(n)
```

```

ratio = invariant_rank / sos_deg
return {
    "metric_name": "Invariant Rank to SOS Degree Ratio",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": abs(ratio - 0.879) < 0.5,
    "counterexample": f"n={n}, invariant_rank={invariant_rank}, sos_degree={sos_deg}" if not c
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_ratio = math.sqrt(sum((r["metric_value"] - mean_ratio) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={result['instances_tested']}, invariant_rank={result['invariant_rank']}, sos_degree={result['sos_degree']}\" if not conjecture_holds")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cc8b5b28.py", line 56, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cc8b5b28.py", line 48, in run_trial
    "counterexample": f"n={n}, invariant_rank={invariant_rank}, sos_degree={sos_deg}" if not conjecture_holds
    ^^^^^^^^^^^^^^^^^
NameError: name 'conjecture_holds' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and provides the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14895
2	propose	ollama_remote	glm4:latest	0	0	14208
3	preregistration	ollama_remote	glm4:latest	0	0	9343
4	novelty	ollama_remote	glm4:latest	0	0	8808
5	novelty	ollama_remote	glm4:latest	0	0	9146
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25406
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9297
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7275
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6932
10	judge	ollama_remote	glm4:latest	0	0	12750

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118060 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/9d3b0b0b8113.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9d3b0b0b8113.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9d3b0b0b8113.tar.gz (if generated)